Group #28 Duck Tape
April Wang (aw764), Emily Felts (ef438), Sabrina Liu (sl2222)

MAE 3780 Robot Project Final Report

I. Robot Design and Strategy Overview

The robot's mechanical design includes a fixed, 3-walled, rectangular outer boundary to trap the cubes from the back and sides. These walls were laser cut from acrylic (Appendix C, Figure 3), since acrylic is stronger than many other materials and much cheaper than 3D printing, and attached to the chassis using standoffs. The provided wheels were swapped out for larger wheels that raised the main chassis of the robot to an appropriate height for blocks to fit underneath the servos.

To keep our design simple and reduce risk of parts breaking or failing, we opted to not expand our footprint during the competition or use power-operated mechanical functions for cube collection. We also chose not to include a color sensor so that we could focus our efforts on other aspects of the strategy. A color sensor would have been placed as close to the ground as possible, which limited the space below the main body for blocks.

A platform (Appendix C, Figure 4) composed of multiple laser cut parts that were glued together was also made for the breadboard to sit stably on top of the Arduino's mini breadboard and for the ultrasonic sensor, which we had originally considered using, to attach to.

We built a spoon launcher to launch a rubber duck onto the arena floor to disrupt the opponent robot. The launcher stores potential energy through a rubber band wrapped around a laser cut acrylic rectangular pole. A plastic spoon launched the duck forward. The launcher was activated with a hobby servo that was set to rotate a small amount at the beginning of the match. The servo arm was extended with a small laser cut acrylic block to reach the tip of the spoon and hold the spoon in place until it was launched.

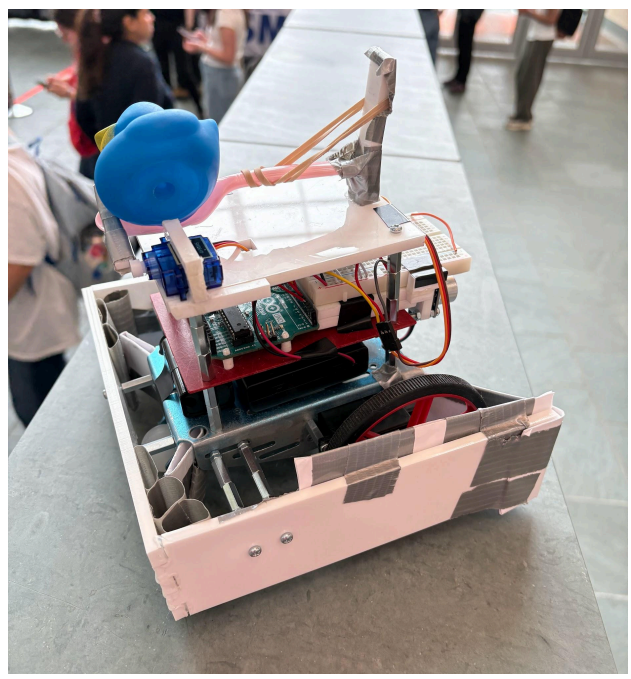


Figure 1: Image of final robot

Our strategy was to launch the rubber duck at the start of the match, then travel in a fixed path, as shown in Appendix D, Figure 10. Since no color/QTI sensors

were used, the path and timing delays were hardcoded. The rationale was to maximize coverage of the cube area while getting off the board as quickly as possible to avoid interference from the opponent robot.

II. Design Process Reflection

Throughout testing, we found the robot could get immobilized due to cubes obstructing the unprotected wheels. We decided to add paper shields to prevent cubes from interfering with the wheels. Also considering the available space for cube collection, we changed the mounting of the wheel servos to move the wheels outward, so the space in between the wheels can be maximized. We also added paper to prevent cubes from coming out from under the acrylic walls if we came off the board and added duct tape loops to the interior face of the rear barrier to stick cubes to the wall. This allowed some cubes to be carried by the robot in the event the robot went outside the edge of the arena.

When finalizing our strategy, the ultrasonic sensor was abandoned. It was initially used for collision avoidance and performed well. However, since that relied on random turns, given that no color/QTI sensors were used, this approach introduced additional uncertainty, which could risk the robot to keep traveling outside of the cube area. The run time for our code was also quite short, so we decided that the ultrasonic sensor would have been more beneficial for a longer program.

For the duck launcher, we decided to laser cut a small bridge to connect the servo mounts (see Appendix C, Figure 6) above the hobby servo. This allowed us to glue around the servo without putting glue directly onto the servo in order to preserve the reusability of the servo, as in alignment with competition guidelines. For the laser cut parts, we ended up using hot glue to bind the acrylic pieces together since super glue was not available.

III. Competition Analysis

During the competition, our robot performed about as expected. Since we chose to go in one direction, there was about a 50% chance the opponent robot would go in that same direction, causing a big disruption in our strategy. This correlated with us winning about 50% of our matches.

One thing unexpected is our strategy aimed at arriving at the middle of the competition board and starting cube collection as quickly as possible, to minimize the chance of colliding with the opponent. However, we observed a nontrivial delay between resetting the robot and activating the launcher, which caused our robot to become entangled with the opponent more frequently than expected.

For strengths, our robot was very sturdy, and most of the cubes collected initially stayed inside the robot even when we collided with others. As mentioned, our weaknesses included hard coding our path, as well as stopping our movement very early on. Since we had intended to get off the board, we wanted our robot to stop early, but once we collided with other robots, we were stuck on the board, unable to continue moving and pick up more cubes.

We are a little disappointed that no opponent collected our duck.

IV. Conclusions

Overall, we are pleased with what we were able to accomplish given the constraints. If we could redo the project, we would make our robot move for a longer period of time and opt for line detection. During the competition, we found that the robots that did well were able to keep circulating the arena for the entire duration of the match. Ours stopped after a short amount of time, so we were only able to collect the cubes intercepted during the first pass. We would keep our 3-walled boundary box design and as it was much larger than many of the smaller robots. This and our raised robot design gave us a lot of space for cubes. Additionally, primarily laser cutting parts and making parts out of paper/duck tape was cheap and effective, so we would also do that again.

For students working on this project next year, it is important to remember that you actually know more than you think. All of the labs will come in handy, and you can figure it out. Redo the labs if you need to. Even though you may have hated this class at some point during the semester, many of the things you learned come together for this class. Take a deep breath and look at your code from labs 3 & 4.

Additionally, we encourage students to have fun while designing their robot, as it makes the process much less tedious and stressful. Although the duck did not add much to our robot (probably even hindered its success), we really enjoyed making the duck launching design, and the TAs and students around us thought it was fun as well.

Appendix A: Bill of Materials

Part	Description	Vendor	Item Number	Price per item	Quantity	Total Cost	Link to Item
Big Red Wheels	3.5" D	Lab	N/A	$2	2	$4.00	N/A
RPL Laser Cutting	Laser cutting parts in RPL using 1/4" thick 12" x 12" acrylic provided by MAE	MAE, RPL	N/A	$0.05/inch cut + $0.50/part + $5/sheet	12" x 12" cut 16 parts 1 sheet	$20.20	N/A
Rubber duck	Small plastic duck	Borrowed from roommate	N/A	$1	1	$1.00	N/A
Spoon	Plastic spoon used to launch duck	Taken from plastic spoon stash	N/A	$0.05	2	$0.10	N/A
					Total Cost	$25.30	

Table 1: Bill of Materials

Appendix B: Circuit Diagram

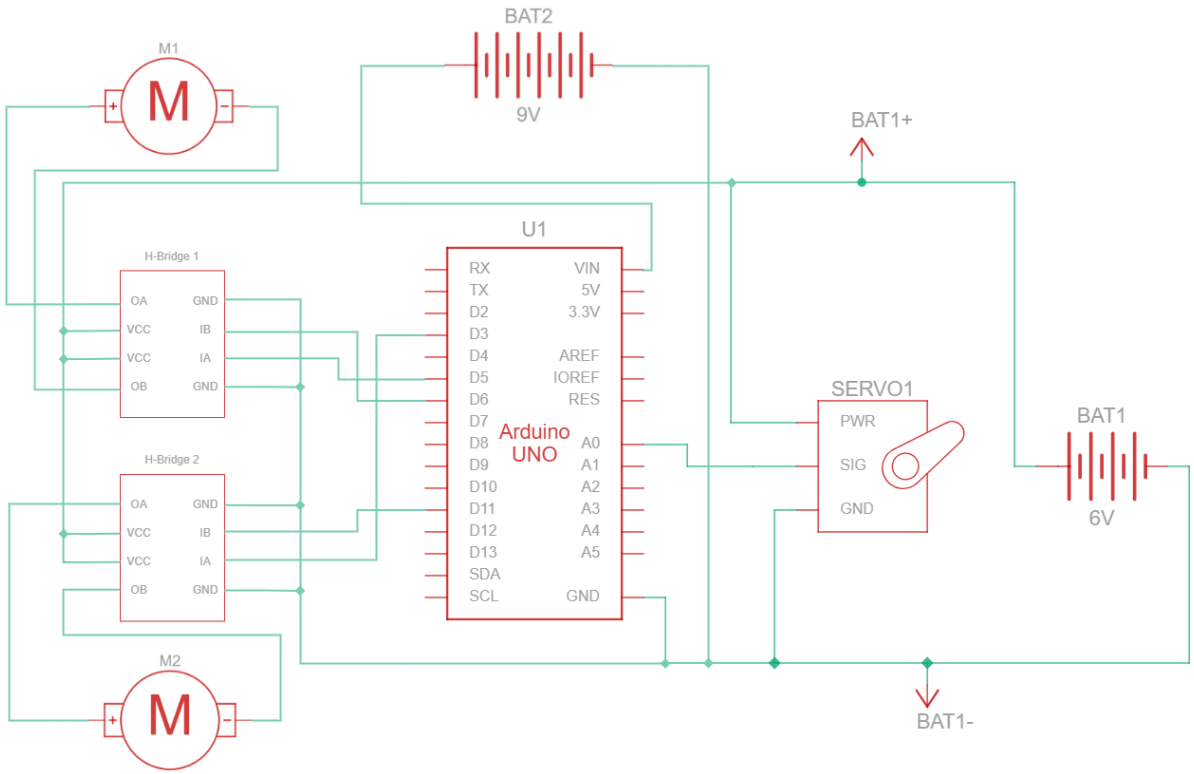


Figure 2: Final circuit diagram

Appendix C: CAD Files and Drawings

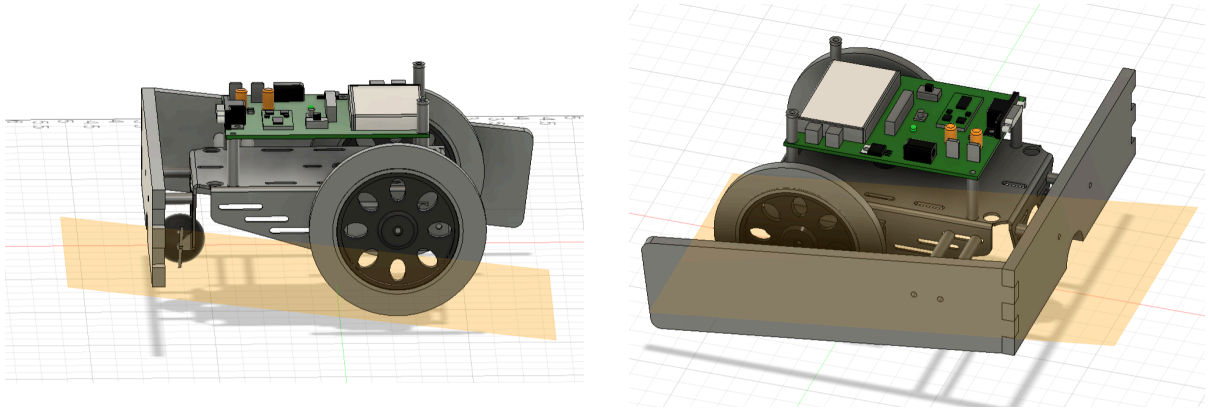


Figure 3: CAD of two of the three ¼” thick walls to be laser cut attached to provided robot CAD

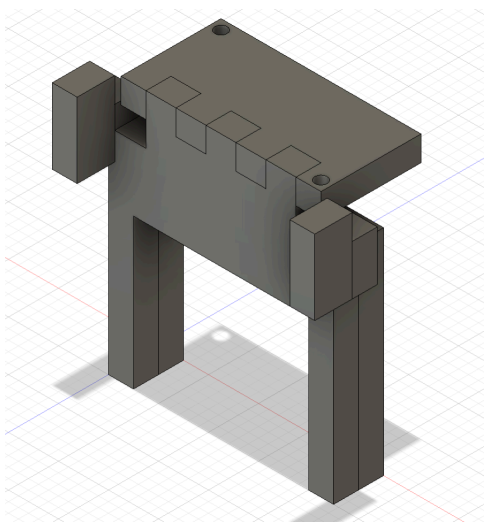


Figure 4: CAD of platform that the breadboard sits on top of and the ultrasonic sensor attaches to the front of. The platform is intended to slot around the Arduino's mini breadboard.

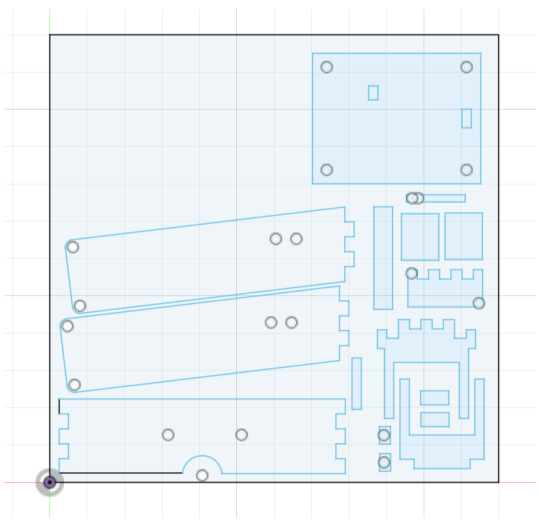


Figure 5: CAD sketch of all the parts to be laser cut in a 12” x 12” square that represents the ¼” thick 12” x 12” acrylic platform used.

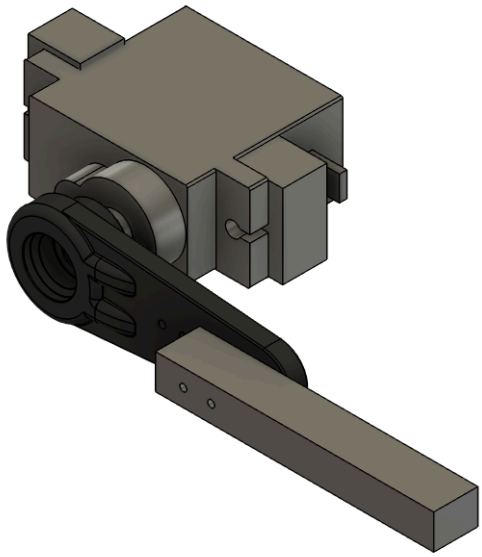


Figure 6: CAD of hobby servo assembly used to launch duck. Servo and black servo arm CADs imported from AutoDesk website. Servo arm extender and mounts behind screw holes made from ¼ “ laser cut acrylic. Servo arm extender connected to servo arm via duct tape. The small servo mounts were hot glued to the platform.

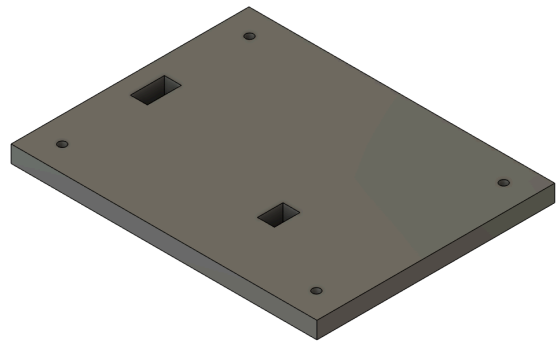


Figure 7: CAD of platform mounted above Arduino, breadboard, and wiring. Made from ¼ “ laser cut acrylic and mounted with M3 standoffs.

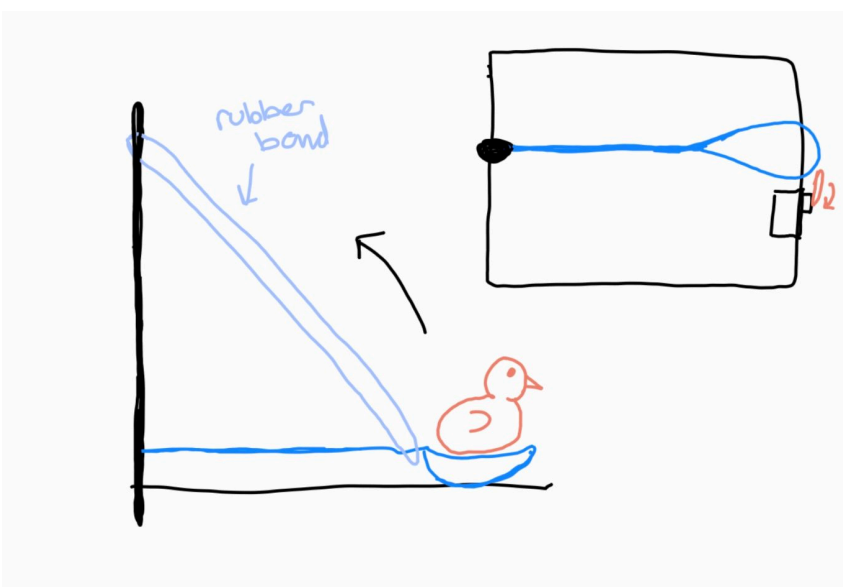


Figure 8: Sketch of overall duck launcher design. Rubber band pulls spoon with duck upwards to launch. Spoon held down by hobby servo arm that rotates a small amount to launch.

Appendix D: Flowchart

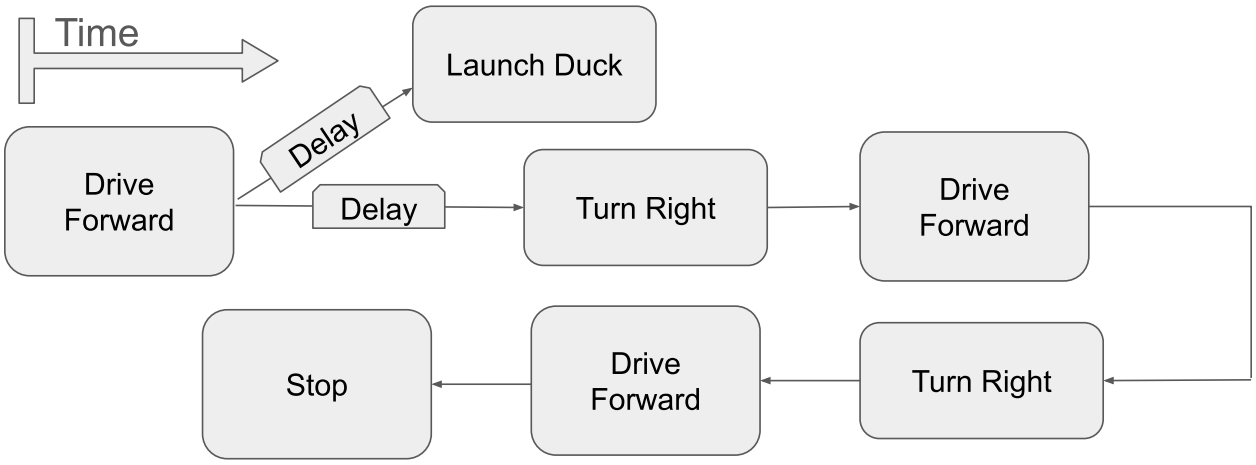
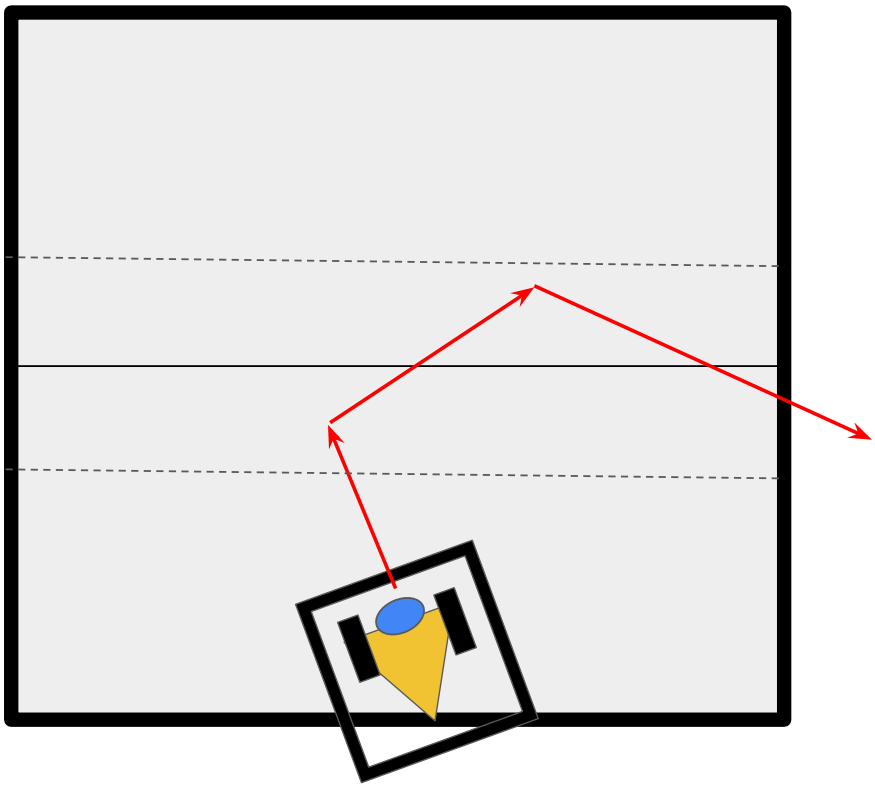


Figure 9, above: Flowchart of code
Figure 10, below: Schematic of hardcoded path.



Appendix E: Code

```
// TIMER0 - motor
// TIMER1 - launcher
// TIMER2 - motor

int main(void){

  // ---------- TIMER0 for motor control ----------
  TCCR0A = 0b10100001; // Phase correct PWM, clear OCR0A,OCR0B on
CompMatch when upcounting
  TCCR0B = 0b00000001; // TOP=0xFF, no prescaling
  TCCR2A = 0b10100001; // Phase correct PWM, clear OCR2A,OCR2B on
CompMatch when upcounting
  TCCR2B = 0b00000001; // TOP=0xFF, no prescaling

  // ---------- Pin I/O config ----------
  // PD3 (OC2B): right motor (+)
  // PB3 (OC2A): right motor (-)
  // PD5 (OC0B): left motor (+)
  // PD6 (OC0A): left motor (-)
  // PB1 (OC1A): launcher
  DDRB = 0b00001010;
  DDRD = 0b01101000;
  DDRC = 0b1;

  set_motor(0,0); // stop initially

  // ---------- TIMER1 for launcher ----------
  TCCR1A = 0b10100010; // Phase correct PWM, clear OCR1A, OCR1B on
CompMatch when upcounting
  TCCR1B = 0b00010011; // prescaler 64
  // period (ns) = 2 * ICR1 * prescaler * 62.5
  ICR1 = 2500;
  // pulse width (ns) = (OCR1A + 1) * prescaler * 62.5
  OCR1A = 80; // rotate
  _delay_ms(300);
  OCR1A = 190; // stop
```

```
  // ---------- Zig-zag movement ----------
  set_motor(255,255); //fwd
  _delay_ms(1300);
  set_motor(255,-255); //right
  _delay_ms(500);
  set_motor(255,255); //fwd
  _delay_ms(1700);
  set_motor(255,-255); //right
  _delay_ms(400);
  set_motor(255,255); //fwd

  _delay_ms(5000);
  set_motor(0,0);

}

// duty_L, duty_R should be between [-255, 255]
void set_motor(int duty_L, int duty_R){
  if (duty_L >= 0){
    OCR0B = duty_L; // left motor (+) high
    OCR0A = 0;      // left motor (-) low
  } else {
    OCR0A = -duty_L; // left motor (-) high
    OCR0B = 0;       // left motor (+) low
  }
  if (duty_R >= 0){
    OCR2B = duty_R; // right motor (+) high
    OCR2A = 0;      // right motor (-) low
  } else {
    OCR2A = -duty_R; // right motor (-) high
    OCR2B = 0;       // right motor (+) low
  }
}
```